

1) Visao geral do projeto

Este projeto e uma aplicacao web local para monitorar a comunicacao de relgios de ponto de varias empresas, separadas por sistema:

- DIMEP
- MADIS (MDComune)

A aplicacao foi desenhada para uso operacional diario com foco em:

- importacao simples de empresas por planilha (Excel/CSV)
- consulta em lote das APIs externas
- visualizacao clara por empresa e por relgio
- regras de negocio para evitar falso positivo de comunicacao
- tratamento de erro por empresa (sem derrubar o lote inteiro)

A versao ativa atual roda em Node.js + Express no backend e HTML/CSS/JS no frontend.

2) Objetivo funcional

Responder rapidamente estas perguntas:

- 1 Quais empresas estao com todos os relgios comunicando?
- 2 Quais empresas tem relgios sem comunicacao?
- 3 Quais relgios estao com falha?
- 4 Qual foi a ultima coleta de cada relgio em horario de Brasilia?

3) Arquitetura tecnica

3.1 Componentes

- Frontend estatico:
- public/index.html
- public/styles.css
- public/app.js
- Backend HTTP local:
- server.js (Express)
- Script de inicializacao local:
- run-local.ps1

3.2 Fluxo de alto nivel

```
flowchart LR
    A[Usuario no navegador] --> B[Frontend app.js]
    B --> C[Backend local Express server.js]
    C --> D[API DIMEP]
    C --> E[API MADIS]
    C --> B
    B --> A
```

3.3 Por que a chamada externa e feita no backend

As APIs externas exigem headers especificos e podem ter bloqueios de borda/CORS/anti-bot.

Com a chamada no backend local:

- evita CORS no navegador
- protege chave API de ficar exposta na tela/rede do browser
- permite normalizar respostas diferentes entre DIMEP/MADIS
- permite cache local curto para reduzir carga

4) Estrutura de pastas e arquivos

```
PainelMonitoria/  
|-- node_modules/  
|-- public/  
|   |-- index.html  
|   |-- styles.css  
|   |-- app.js  
|-- server.js  
|-- package.json  
|-- package-lock.json  
|-- run-local.ps1  
|-- README.md  
|-- DOCUMENTACAO_COMPLETA.md  
|-- main.py           # legado (versao Python)  
|-- requirements.txt  # legado (versao Python)
```

Observacao:

- `server.js` e o backend oficial atual.
- `main.py` e legado da implementacao anterior (FastAPI). Pode ser removido futuramente se nao houver necessidade de rollback.

5) Requisitos e execucao local

5.1 Requisitos

- Windows com Node.js 20+
- NPM disponivel

5.2 Execucao padrao

No diretorio do projeto:

```
npm install  
npm start
```

A aplicacao sobe em:

- `http://127.0.0.1:8000`

5.3 Execucao PowerShell assistida

Se node/npm nao estiver no PATH:

```
.\run-local.ps1
```

Se politica de execucao bloquear scripts:

```
powershell -ExecutionPolicy Bypass -File .\run-local.ps1
```

6) Integracoes externas

6.1 Sistemas suportados

- DIMEP: <https://www.dimepkairos.com.br>
- MADIS: <https://www.mdcomune.com.br>

6.2 Endpoint usado para ambos

- POST /RestServiceApi/Clock/SearchClocks

Body enviado pelo backend:

```
{  
  "TodosRelogios": true  
}
```

Headers principais enviados:

- identifier: CNPJ da empresa (com fallback de formatos)
- key: API key da empresa

Headers adicionais de compatibilidade (para reduzir erro 403):

- User-Agent
 - Referer
 - Origin
 - Accept-Language
-

7) Modelo de dados interno

7.1 Empresa em memoria (backend)

Campos:

- id
- name
- identifier
- api_key (sensível, não retorna ao frontend)
- system (DIMEP ou MADIS)

7.2 Empresa publica (frontend)

Campos retornados:

- id
- name
- identifier
- system

7.3 Relogio normalizado

Campos:

- code
 - name
 - ip
 - ip_is_null
 - last_collection_brt
 - is_communicating
 - is_disabled
-

8) Importacao da planilha

8.1 Formatos aceitos

- .xlsx
- .xls
- .csv

Limite de upload:

- 10 MB

8.2 Colunas obrigatorias

A aplicacao aceita variacoes de cabecalho por aliases. Conceitualmente, precisa de:

- Nome da empresa
- CNPJ (identifier)
- API Key
- Sistema

Exemplos de aliases aceitos:

- Nome: Nome da empresa, Empresa, Nome, Razao social
- CNPJ: CNPJ, identifier, identificador
- API key: API Key, apikey, key, chave
- Sistema: Sistema, Fornecedor, Plataforma, API

8.3 Validacoes na importacao

Durante importacao:

- linhas incompletas sao ignoradas com warning
- sistema invalido (nao DIMEP/MADIS) gera warning
- duplicidades sao ignoradas (combina sistema + cnpj + api_key)
- se nenhuma linha valida, retorna erro 400

Resultado:

- base em memoria e substituida pela nova planilha

- cache de status e limpo
-

9) Regras de negocio de monitoria

9.1 Regra de relógio desativado

Qualquer relógio com indicativo de desativado (`RelógioDesativado` e aliases) é removido da análise.

9.2 Regra de ultima coleta > 1 hora

Se a ultima coleta do relógio tiver mais de 1 hora de atraso em relação ao horário atual, o relógio é marcado como:

- Sem comunicação

Mesmo que exista algum indicativo externo de comunicação, essa regra prevalece para monitoria operacional.

9.3 Conversão de data/hora

Datas vindas da API são interpretadas como UTC quando necessário e exibidas em:

- `America/Sao_Paulo (BRT)`
- formato: `dd/mm/aaaa hh:mm:ss`

9.4 Quando a API não manda flag de comunicação

Se o campo booleano de comunicação não existir:

- fallback: considera comunicando quando existe data válida de ultima coleta
- depois aplica a regra de 1 hora (que pode virar sem comunicação)

9.5 Status da empresa

A empresa fica ok quando:

- existe ao menos 1 relógio ativo
- e nenhum relógio ativo está sem comunicação

A empresa fica error quando:

- não existe relógio ativo
 - ou existe pelo menos 1 relógio sem comunicação
 - ou houve falha HTTP/rede na consulta da API da empresa
-

10) Regra especial por empresa (API key específica)

Existe regra dedicada para a API key:

- `11b345c7-6790-4df6-a0fb-7b4bee3a2447`

Regras aplicadas **somente** para essa empresa:

- ¹ Bloqueio por código de relógio (`blocked_codes`):
- os códigos listados não aparecem no painel

¹ Filtro por IP nulo (`only_null_ip = true`):

- apenas relógios com IP nulo/vazio aparecem

Isso não afeta as outras empresas.

11) Cache e desempenho

11.1 Cache local

TTL atual:

- 60 segundos por empresa

Chave de cache:

- `system + identifier + api_key`

Comportamento:

- Puxar Status: usa cache quando válido
- Atualizar ignorando cache: força nova consulta externa

11.2 Benefícios

- menos chamadas repetidas para APIs externas
 - menor tempo de resposta em consultas sequenciais
 - menor risco de rate-limit/bloqueio temporário
-

12) API local (backend)

12.1 Health

GET `/api/health`

Resposta:

```
{ "status": "ok" }
```

12.2 Template de planilha

GET `/api/template/companies`

Retorna arquivo `modelo_empresas.xlsx` para download.

12.3 Listar empresas em memória

GET `/api/companies`

Resposta (resumo):

```
{
  "total": 2,
  "grouped": { "DIMEP": [], "MADIS": [] },
  "companies": [
    {
      "id": "dimep-12345678000190-1",
      "name": "Empresa X",
      "identifier": "12.345.678/0001-90",
      "system": "DIMEP"
    }
  ]
}
```

```
}  
]  
}
```

12.4 Importar empresas

POST /api/import-companies

- multipart/form-data
- campo de arquivo: file

Resposta de sucesso:

```
{  
  "total": 10,  
  "warnings": ["Linha 6: empresa duplicada ignorada."],  
  "grouped": { "DIMEP": [], "MADIS": [] },  
  "companies": []  
}
```

Erros comuns:

- 400 arquivo invalido
- 400 formato nao suportado
- 400 arquivo vazio
- 400 arquivo > 10MB
- 400 colunas obrigatorias ausentes

12.5 Puxar status

POST /api/pull-status

Body:

```
{  
  "force_refresh": false  
}
```

Resposta de sucesso:

```
{  
  "total": 10,  
  "updated_at": "25/04/2026 20:20:11",  
  "summary": {  
    "healthy_companies": 7,  
    "unhealthy_companies": 3,  
    "from_cache": 5  
  },  
  "grouped": {  
    "DIMEP": [],  
    "MADIS": []  
  },  
  "companies": [  
    {  
      "id": "...",  
      "name": "...",  
      "identifier": "...",  
      "system": "DIMEP",  
      "status": "ok",  
      "error": null,  
      "from_cache": false,  
      "active_clock_count": 12,  
    }  
  ]  
}
```

```
        "communicating_count": 11,  
        "not_communicating_count": 1,  
        "clocks": [],  
        "updated_at": "25/04/2026 20:20:11"  
    }  
]  
}
```

13) Frontend (comportamento e estado)

13.1 Estado principal em app.js

- companies: empresas importadas
- statusesById: mapa de status por company.id
- selectedCompanyId: empresa aberta no modal
- modalFilter: all, online, offline
- selectedSystem: aba ativa (DIMEP ou MADIS)

13.2 Fluxo principal de uso

- 1 Usuario importa planilha
- 2 Frontend chama /api/import-companies
- 3 UI atualiza contadores e lista de empresas
- 4 Usuario clica em Puxar Status
- 5 Frontend chama /api/pull-status
- 6 UI atualiza:
 - cards de status geral
 - cards de empresas
 - detalhe no modal por relógio

13.3 Modal de detalhes da empresa

No modal, o frontend mostra relógios em blocos com:

- código
- nome
- IP
- última coleta (BRT)
- status visual (comunicando / sem comunicação)

Filtros disponíveis:

- Todos
 - Comunicando
 - Sem comunicação
-

14) Estilo visual e UI

A interface usa tema escuro com paleta azul inspirada em Windows, com efeito glass:

- fundo com gradientes e brilho difuso
- paineis translúcidos (panel-glass)
- contraste alto para leitura operacional
- layout responsivo

Pontos recentes de UX aplicados:

- painel Status geral mais compacto
- remoção de descrição redundante nesse painel
- remoção de legenda para ganhar espaço
- alinhamento de altura entre os painéis superiores

15) Tratamento de erros

15.1 Nível API externa por empresa

Cada empresa é processada de forma isolada.

Se uma falhar (403, timeout, rede), as outras continuam sendo processadas.

15.2 Erros mapeados para o usuário

Exemplos:

- Falha HTTP na API (403): ...
- Não foi possível conectar com a API da empresa.
- Resposta da API inválida (JSON malformatado).

15.3 Erros de importação

Erros de arquivo/colunas retornam 400 com mensagem explícita para correção rápida da planilha.

16) Diagnóstico e troubleshooting

16.1 Erro npm ERR! enoent ... package.json

Causa comum:

- comando executado na pasta errada

Solução:

- entrar em C:\Users\Arthur.saldanha\Documents\Euro\Dev\PainelMonitoria
- rodar `npm install` e `npm start`

16.2 Erro 403 na API

Checklist:

- 1 validar identificador (cnpj) e key da empresa
- 2 confirmar sistema correto (DIMEP/MADIS)
- 3 validar se chave tem permissão no ambiente da API

- 4 testar no Swagger do fornecedor
- 5 lembrar que o backend tenta variacoes de CNPJ (raw, so digitos, formatado)

16.3 Relogio aparece sem nome

Significa que a API nao retornou nome em nenhum campo conhecido (aliases).

Fallback aplicado:

- Relogio sem nome

16.4 Divergencia de comunicacao entre MDComune/Kairos e painel

Causas comuns:

- ultima coleta esta antiga (regra de 1 hora)
- relógio filtrado por regra especial
- relógio desativado removido do painel
- diferenca de timezone/origem da data

17) Seguranca

17.1 Protecao basica de credenciais

- API key fica no backend em memoria
- frontend nao recebe api_key

17.2 Cuidados recomendados para evolucao

- adicionar autenticao para uso do painel
- adicionar criptografia em repouso se persistir dados em disco
- registrar auditoria de importacao e consulta
- limitar origem/rede de acesso (uso interno)

18) Performance

Ja aplicado:

- cache curto por empresa
- parse robusto com aliases
- normalizacao unica por relógio

Melhorias futuras recomendadas:

- limite de concorrencia configuravel por ambiente
- retries com backoff para timeout
- persistencia opcional (SQLite/Postgres) para historico
- pagina de observabilidade (tempo medio por empresa, taxa de erro)

19) Limitacoes atuais

- dados em memoria (reiniciar servidor limpa empresas/status)
 - sem autenticao de usuario
 - sem historico temporal persistente
 - sem testes automatizados no pipeline
-

20) Roadmap sugerido

- 1 Persistencia local (SQLite) para manter base entre reinicios
 - 2 Login e perfis de acesso
 - 3 Exportacao de relatorio CSV/PDF
 - 4 Alertas (Teams/Email) para empresa com falha
 - 5 Empacotar como desktop (Electron) para instalador Windows
-

21) Guia de operacao diaria (didatico)

21.1 Primeiro uso

- 1 abrir o sistema
- 2 clicar em Baixar modelo Excel
- 3 preencher empresas e credenciais
- 4 importar planilha
- 5 clicar em Puxar Status
- 6 abrir empresa para ver relatorios

21.2 Rotina recomendada

- usar Puxar Status para leitura rapida
- usar Atualizar ignorando cache quando precisar validacao imediata
- investigar apenas empresas em vermelho primeiro

21.3 Interpretacao rapida dos numeros

- Comunicando: total de relatorios ativos com status ok
 - Sem comunicacao: total de relatorios ativos com falha
 - Empresas com erro: quantidade de empresas com status final error
-

22) Referencia de campos e aliases

22.1 Aliases de empresa

Mapeamentos relevantes no backend:

- Nome: nome da empresa, empresa, nome, razao social, razaosocial
- CNPJ/identifier: cnpj, identifier, identificador
- API key: api key, apikey, key, chave, chave api, chaveapi
- Sistema: sistema, fornecedor, plataforma, api

22.2 Aliases de relógio

Mapeamentos relevantes:

- Desativado: `relogiodesativado`, `isdisabled`, `disabled`
 - Código: `codigo`, `cod`, `clockid`, `idrelogio`, `id`, `numero`
 - Nome: `nome`, `nomerelogio`, `descricao`, `clockname`
 - IP: `ip`, `enderecoip`, `iprelogio`, `host`
 - Última coleta: `ultimacoleta`, `lastcollection`, `ultimacomunicacao`
 - Comunicação: `comunicando`, `statuscomunicacao`, `iscommunicating`, `online`
-

23) Contrato visual esperado

Empresa card:

- nome + CNPJ
- bolinha verde/vermelha
- ativos
- resumo de comunicando/sem comunicacao

Modal empresa:

- título e CNPJ
 - filtros por status
 - blocos de relógio com campos operacionais
-

24) Boas praticas para manutencao do codigo

- evitar alterar regras globais para resolver excecao de 1 empresa
 - usar `SPECIAL_COMPANY_RULES` para regras dedicadas
 - manter mensagens de erro curtas e objetivas para operacao
 - não enviar `api_key` para frontend
 - validar impacto de `timezone` em qualquer mudanca de data
-

25) Testes manuais recomendados (checklist)

25.1 Importacao

- [] importar planilha valida com DIMEP e MADIS
- [] importar planilha com coluna faltando e validar erro
- [] importar arquivo vazio e validar erro
- [] importar duplicidade e validar warning

25.2 Consulta de status

- [] puxar status com cache (2x seguidas)
- [] puxar status sem cache (`force_refresh=true`)
- [] validar contador de cache no painel

- ☐ validar uma empresa com erro 403

25.3 Regras de negocio

- ☐ relógio desativado não aparece
- ☐ relógio com coleta >1h aparece sem comunicação
- ☐ data no modal está em BRT
- ☐ regra especial por API key aplicada

25.4 UI

- ☐ painel esquerdo e direito com proporção equilibrada
- ☐ modal abre/fecha por clique e Esc
- ☐ responsividade em largura menor

26) Como evoluir para software instalável Windows

Viabilidade: alta.

Caminho prático:

- 1 manter backend Node local
- 2 encapsular frontend + backend com Electron
- 3 gerar instalador .exe com electron-builder

Ganhos:

- ícone/app dedicada
- instalação padrão Windows
- inicialização simplificada para usuário final

27) Resumo executivo

O projeto entrega uma operação local robusta para monitoria de relógios DIMEP/MADIS com:

- importação simples por Excel
- consulta em lote com tratamento de falhas por empresa
- regras de negócio operacionais (desativado, stale >1h, timezone)
- regra especial por empresa específica
- dashboard visual claro para ação rápida

A base técnica atual está preparada para manutenção e evolução incremental, com boa separação entre frontend e backend.